

Java 15 — Text Blocks (Final)

Text Blocks became a **final, standard Java feature in Java 15 (2020)** after previewing in Java 13 and Java 14. They provide a cleaner way to represent multi-line string literals without excessive escaping and concatenation.

Aspect	Details
Java Version	Java 15
Year	2020
Feature	Text Blocks — Final
UniqueClassName	TextBlocksFinal
JEP	JEP 378
Primary Purpose	Make multi-line strings easier to write and maintain
Syntax	""" ... """
Alternative	Traditional string literals using "...", concatenation, or StringBuilder

1. Basic Example

Before Java 15:

```
String json = "{\n" +  
    "  \"name\": \"Anurag\", \n" +  
    "  \"age\": 30\n" +  
    "}";
```

Java 15 Text Block:

```
String json = """  
    {  
      "name": "Anurag",  
      "age": 30  
    }  
    """;
```

The second version is much closer to the actual JSON structure.

2. Advantages and Disadvantages

Advantages	Disadvantages
Much more readable for multi-line strings	Not useful for ordinary one-line strings
Reduces <code>\n</code>, <code>\</code>, and concatenation	Indentation rules can initially be confusing
Excellent for JSON, XML, SQL, HTML, templates, etc.	Developers must understand incidental whitespace handling
Easier to maintain large embedded strings	Can accidentally introduce unwanted whitespace
Makes source code visually resemble the resulting text	Dynamic values still require concatenation, formatting, or templating
Reduces string-concatenation boilerplate	Not a replacement for dedicated template engines
Easier to copy/paste structured text	Very large embedded resources may be better stored as external files
Supports normal String operations	Does not provide automatic validation of JSON/XML/SQL syntax
Compile-time string literal, so no runtime file-loading overhead	Large literals still increase class-file/application size
Supports escape sequences when required	Existing code style may prefer conventional strings for short values

3. When to Use Text Blocks

Situation	Should You Use?	Example
Multi-line JSON	Yes	REST request/response examples
Multi-line SQL	Yes	Complex SELECT queries
XML documents	Yes	SOAP/XML configuration
HTML templates/snippets	Yes	Generating small HTML fragments
Markdown content	Yes	Embedded documentation
Multi-line test data	Yes	Unit/integration tests

GraphQL queries	Yes	GraphQL API requests
Regular expressions spanning multiple lines	Sometimes	Complex regex definitions
Configuration snippets	Yes	YAML/properties-like content
One-line messages	No	"Order completed"
Simple constants	No	"ACTIVE"
Very large static documents	Usually no	Store them as external resources
User-generated content	No	Use external input/storage
Complex dynamic templates	Usually no	Use a template engine

4. Real-Time Use Cases

Use Case	Traditional Approach	Text Block Approach	Benefit
JSON API payload	Escaped quotes + \n	Natural JSON structure	Easier to read
SQL query	String concatenation	Multi-line SQL	Easier query maintenance
HTML email	Numerous escaped strings	HTML directly inside text block	Better readability
XML/SOAP request	Escaped XML	XML-like structure	Easier to understand
GraphQL	Concatenated query strings	Multi-line query	Cleaner API code
Unit-test expected output	"\n" everywhere	Expected output visually preserved	Easier assertions
CLI help text	Multiple concatenated strings	One text block	Less boilerplate
Generated source code	Escaped Java code	Embedded source-like text	Easier maintenance

SQL migration scripts	Concatenated SQL	Readable SQL	Better developer experience
Documentation snippets	String concatenation	Markdown/code block	Easier editing

5. Real-Time Examples

JSON

```
String requestBody = """
    {
      "customerId": 101,
      "product": "Laptop",
      "quantity": 2
    }
    """;
```

Useful in a REST client when constructing a **small static JSON payload**.

SQL

```
String sql = """
    SELECT id, name, email
    FROM customers
    WHERE status = 'ACTIVE'
      AND country = 'IN'
    ORDER BY name
    """;
```

This is significantly easier to maintain than:

```
String sql = "SELECT id, name, email " +
    "FROM customers " +
    "WHERE status = 'ACTIVE' " +
    "AND country = 'IN' " +
    "ORDER BY name";
```

For dynamic values, however, prefer **prepared statements** rather than inserting values into the text block.

HTML

```
String html = """
```

```
<html>
  <body>
    <h1>Welcome</h1>
    <p>Your order has been confirmed.</p>
  </body>
</html>
""";
```

Useful for small HTML fragments, emails, or test fixtures.

Unit Testing

String expected = ""

```
{
  "status": "SUCCESS",
  "code": 200
}
""";
```

This makes expected multi-line output much easier to compare with actual output.

6. When Not to Use Text Blocks

Situation	Why Avoid It?	Better Choice
Single-line string	Text block adds unnecessary syntax	Normal string literal
Highly dynamic content	Text blocks aren't a templating system	Template engine / formatting
Huge HTML document	Makes Java source unnecessarily large	External HTML resource
Huge SQL scripts	Difficult to manage in Java source	.sql resource files
User-provided data	Shouldn't be embedded into source	Database/file/request data
Localization messages	Text blocks don't solve localization	ResourceBundle / i18n system
Structured data requiring validation	Text blocks are just strings	JSON/XML libraries

SQL with variable parameters	String interpolation can cause SQL injection	PreparedStatement

7. Alternate Feature

Text Blocks Final	Alternative	When Alternative Is Better
Java 15 Text Blocks	Traditional String literals	Short/simple strings
Java 15 Text Blocks	StringBuilder	Programmatically constructing strings
Java 15 Text Blocks	String.format() / formatted()	Small strings requiring substitution
Java 15 Text Blocks	Template engines	Dynamic HTML/email/web templates
Java 15 Text Blocks	External resource files	Large SQL, HTML, XML, Markdown, etc.
Java 15 Text Blocks	JSON libraries such as Jackson	Complex/dynamic JSON
Java 15 Text Blocks	XML libraries	Complex XML generation/parsing

Important distinction

A Text Block is **not a template engine**.

For example:

```
String name = "Anurag";
```

```
String message = ""
    Hello, %s!
    Welcome to our application.
    """".formatted(name);
```

Here the **Text Block** provides the multi-line literal, while formatted() supplies the dynamic value.

8. Text Blocks vs Traditional Strings

Feature	Traditional String	Text Block
Introduced	Earlier Java versions	Java 13 preview
Finalized	—	Java 15
Multi-line support	Awkward	Excellent
Syntax	"..."	"""..."""
Newlines	Usually \n	Natural line breaks
Quotes	Often escaped	Many quotes need no escaping
Readability	Lower for large text	Higher
JSON/XML/SQL	Verbose	Very convenient
Dynamic substitution	Concatenation/formatting	Still requires formatting/concatenation
Large external content	Not ideal	Still not ideal
Runtime type	String	String

Bottom line

Use `TextBlocksFinal` whenever a string is naturally multi-line and its readability matters. It is particularly valuable for **JSON, SQL, XML, HTML, GraphQL, test fixtures, and embedded documentation**.

For **short strings, highly dynamic content, or very large documents**, use normal strings, formatting/template engines, or external resource files instead.

Java 15 — Hidden Classes

Hidden Classes were introduced as a **final feature in Java 15 (2020)** through **JEP 371**. They are classes that are intended primarily for use by frameworks and runtime-generated code and are **not discoverable by normal application code**.

They are particularly useful for **dynamic language implementations, runtime code generation, proxies, bytecode frameworks, and JVM-based frameworks**.

Aspect	Details
Java Version	Java 15
Year	2020
Feature	Hidden Classes
UniqueClassName	HiddenClasses
JEP	JEP 371
Primary Purpose	Support dynamically generated classes that should not be discoverable or treated as ordinary application classes
Main API	MethodHandles.Lookup#defineHiddenClass()
Typical Users	Frameworks, libraries, JVM language runtimes, bytecode-generation tools
Normal Application Usage	Relatively uncommon

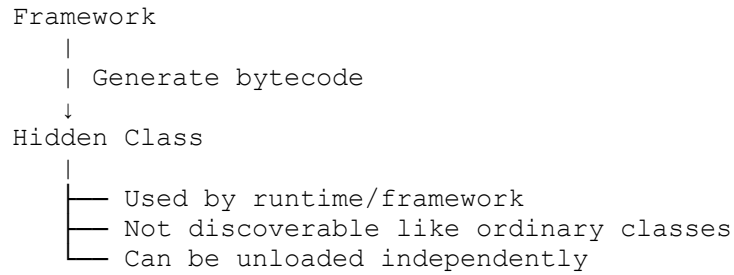
1. What Are Hidden Classes?

A normal class might look like:

```
public class CustomerService {  
    public void process() {  
        System.out.println("Processing customer");  
    }  
}
```

A framework may instead generate bytecode dynamically at runtime. If that generated class does not need to be visible as a normal named class, it can be defined as a **hidden class**.

Conceptually:



A simplified example using the API:

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
```

```
MethodHandles.Lookup hiddenLookup =  
    lookup.defineHiddenClass(  
        classBytes,  
        true,  
        MethodHandles.Lookup.ClassOption.NESTMATE  
    );
```

```
Class<?> hiddenClass = hiddenLookup.lookupClass();
```

The exact bytecode-generation process is omitted here; normally a framework generates classBytes.

2. Advantages of Hidden Classes

Advantage	Explanation	Practical Benefit
Supports runtime-generated classes	Designed for classes created dynamically	Useful for frameworks and code-generation libraries
Better encapsulation	Hidden classes are not intended to be directly discovered by ordinary application code	Reduces exposure of implementation details
Independent unloading	JVM can unload hidden classes when their associated runtime structures are no longer reachable	Helps long-running applications
Useful for dynamic language runtimes	Languages can generate implementation classes dynamically	Supports JVM language implementations

Good for generated implementation details	Framework-generated classes don't need to become part of the application's visible class structure	Cleaner runtime architecture
Works with method handles	Integrates closely with the MethodHandles API	Powerful runtime invocation mechanisms
Supports nestmate relationships	Can be defined as nestmates of the lookup class	Allows access to compatible private members
Avoids permanent class pollution	Large numbers of generated implementation classes don't necessarily behave like ordinary application classes	Helpful for systems generating many classes
Suitable for bytecode frameworks	Bytecode-generation libraries can create implementation classes dynamically	Enables sophisticated runtime optimization

3. Disadvantages of Hidden Classes

Disadvantage	Explanation	Impact
Complex API	Requires understanding MethodHandles, Lookup, bytecode and class loading	High learning curve
Not intended for ordinary business logic	Most applications don't need hidden classes	Adds unnecessary complexity
Requires generated bytecode	You generally need a mechanism to produce valid class bytes	More infrastructure
Harder to debug	Generated/hidden classes can be less straightforward to inspect	Debugging becomes more difficult
Framework-oriented feature	Primarily designed for JVM infrastructure	Limited direct application value
More difficult tooling integration	IDEs, profilers and debugging tools may provide less convenient visibility	Operational complexity

Lifecycle considerations	Incorrect references can prevent generated classes from being unloaded	Potential memory issues
Not a security boundary by itself	Hidden does not mean trusted or magically secure	Must not be treated as a security mechanism
Bytecode knowledge may be required	Advanced use often involves ASM or another bytecode-generation approach	Increased implementation complexity
Can complicate maintenance	Runtime-generated behavior is harder to understand than normal source code	Higher maintenance cost

4. When to Use Hidden Classes

Scenario	Use Hidden Classes?	Why
Runtime-generated implementation classes	Yes	Core intended use case
Dynamic proxy implementation	Yes	Generated implementation need not be normally discoverable
JVM language runtime	Yes	Runtime can generate implementation classes
Bytecode optimization framework	Yes	Runtime-generated optimized implementations
Expression/query compiler	Yes	Compiled expressions can have generated implementation classes
Serialization framework	Yes	Generated serializers/deserializers may benefit
Dependency-injection framework internals	Sometimes	Generated implementation details can remain hidden
ORM-generated implementations	Sometimes	Runtime-generated entity/proxy implementations
Ordinary REST service	No	Normal Java classes are simpler

Business/domain classes	No	No meaningful benefit
Simple utility class	No	Completely unnecessary
Configuration class	No	Use normal classes
DTO/record	No	Use records/classes
Static application logic	No	Hidden classes solve a different problem

5. Real-Time Use Cases

Use Case 1 — Dynamic Proxy Generation

Frameworks frequently need to generate classes at runtime.

For example:

```

Application
↓
Service Interface
↓
Framework
↓
Generate implementation bytecode
↓
Hidden Class
↓
Intercept method call

```

A framework could generate an implementation containing behavior such as:

```

public Object invoke(Object[] args) {
    log();
    securityCheck();
    return target.call(args);
}

```

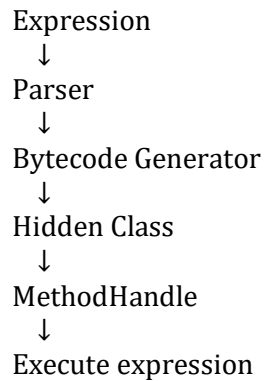
The generated implementation is an **implementation detail**, so making it a normal application-visible class may be unnecessary.

Use Case 2 — Expression Compiler

Imagine an application allowing users to define expressions:

`price * quantity + tax`

A runtime compiler could translate this into JVM bytecode:

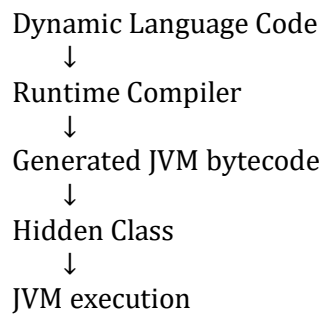


This can be useful for **rules engines, query engines, scripting systems, and expression languages**.

Use Case 3 — JVM Language Implementation

A JVM-based language may dynamically generate classes while executing or compiling code.

For example:



This is one of the use cases that motivated the feature.

Use Case 4 — Serialization Framework

A serialization framework might generate specialized code for:

```
class Customer {  
    String name;
```

```
    int age;  
}
```

Instead of repeatedly using reflection, it could generate optimized serialization code:

```
Customer  
↓  
Runtime analysis  
↓  
Generate serializer bytecode  
↓  
Hidden serializer class  
↓  
Fast serialization
```

The generated serializer is an implementation detail and doesn't necessarily need to become a normal application class.

Use Case 5 — Dependency Injection / AOP Framework

An AOP framework might generate a runtime implementation:

```
Original method  
↓  
Generated implementation  
↓  
Logging  
↓  
Security  
↓  
Transaction  
↓  
Original method
```

The generated implementation could be hidden because application developers don't need to interact with it directly.

6. When NOT to Use Hidden Classes

Situation	Why Not?	Better Choice
Business service	No dynamic-class requirement	Normal Java class
REST controller	Static application	Normal class

	component	
Repository	Usually statically defined	Interface + implementation
DTO	Static data representation	Record/class
Utility methods	No runtime generation	Utility class
Configuration	No benefit	Normal configuration class
Simple proxy	Don't need low-level control	java.lang.reflect.Proxy
Large application architecture	Hidden classes can make debugging harder	Normal classes/modules
Security isolation	Hidden classes aren't a security mechanism	Java security architecture / access control
Small application	Complexity outweighs benefits	Ordinary Java classes

7. Hidden Classes vs Alternatives

Hidden Classes	Alternative	When Alternative Is Better
MethodHandles.Lookup# defineHiddenClass()	java.lang.reflect.Proxy	Standard interface-based dynamic proxies
Hidden classes	Normal Java classes	Classes known at compile time
Hidden classes	MethodHandle	When you only need dynamic method invocation rather than a generated class
Hidden classes	LambdaMetafactory	Generating JVM lambda/function implementations
Hidden classes	Byte Buddy	When you want a higher-level runtime bytecode-generation framework
Hidden classes	ASM	When you need low-level bytecode generation/control
Hidden classes	ClassLoader# defineClass	When the generated class should

	0	behave like a normal loadable class
Hidden classes	invokedynamic	Dynamic call-site and language-runtime scenarios

8. Hidden Classes vs ClassLoader#defineClass()

This is one of the most important comparisons.

Feature	Hidden Class	Normal ClassLoader Class
Visibility	Intended to be hidden/non-discoverable	Normal class visibility
Runtime generation	Excellent	Excellent
Class.forName() style discovery	Not intended	Normal mechanism
Framework implementation details	Excellent	Less suitable
Independent unloading	Strong use case	Depends on class-loader lifecycle
Debugging	More difficult	Easier
Application classes	Usually unnecessary	Best choice
Framework-generated classes	Very useful	Also possible
Dynamic language implementation	Excellent	Possible
Complexity	High	Moderate

9. Hidden Classes vs Dynamic Proxy

Aspect	Hidden Classes	java.lang.reflect.Proxy
Abstraction level	Low-level JVM feature	High-level Java API
Primary purpose	Runtime-generated hidden	Dynamic interface proxy

	implementation	
Bytecode control	High	Low
Ease of use	Difficult	Easy
Custom bytecode	Yes	No direct bytecode control
Interface proxy	Possible	Excellent
Framework development	Excellent	Excellent for simpler cases
Business application	Rarely appropriate	Sometimes useful
Maintenance	More complex	Easier

10. Key Point About Hidden Classes

A useful way to remember the feature is:

Hidden Classes are primarily infrastructure for frameworks and JVM runtimes, not a replacement for ordinary Java classes.

For example:

Normal Java class

↓

Known application type

↓

Used directly by developers

Hidden class

↓

Generated dynamically

↓

Framework/JVM infrastructure

↓

Implementation detail

Final Recommendation

Requirement	Recommendation
Need a normal application class	Normal Java class
Need an interface-based proxy	<code>java.lang.reflect.Proxy</code>

Need dynamic method invocation	Method Handles
Need lambda/function implementation generation	LambdaMetafactory
Need runtime bytecode generation	ASM / Byte Buddy
Need generated classes that should remain hidden implementation details	Hidden Classes
Building a JVM language/runtime	Hidden Classes
Building a sophisticated framework	Consider Hidden Classes
Writing normal business applications	Avoid Hidden Classes

In short: HiddenClasses is a **specialized Java 15 JVM feature** designed mainly for **framework authors, bytecode generators, dynamic language runtimes, and other infrastructure software**. For normal application development, standard classes, interfaces, proxies, method handles, or higher-level bytecode-generation libraries are usually better choices.

Java 15 — Sealed Classes (Preview)

Sealed Classes were introduced as a **preview feature in Java 15 (2020)** through **JEP 360**. They allow a class or interface to explicitly control which other classes or interfaces may extend or implement it.

The feature was subsequently improved through preview iterations and became **final in Java 17**.

Aspect	Details
Java Version	Java 15
Year	2020
Feature	Sealed Classes — Preview
UniqueClassName	SealedPreview
JEP	JEP 360
Status in Java 15	Preview

Finalized	Java 17
Primary Purpose	Restrict inheritance to a known set of permitted types
Main Keywords	sealed, permits
Typical Use	Domain modeling, API design, state machines, controlled hierarchies

1. What Are Sealed Classes?

Normally, a Java class can be extended by any accessible class:

```
class Payment {
}
```

Potentially:

```
class CreditCardPayment extends Payment {
}
```

```
class CashPayment extends Payment {
}
```

```
class UnknownPayment extends Payment {
}
```

A **sealed class** restricts this inheritance:

```
public sealed class Payment
    permits CreditCardPayment, CashPayment {
}
```

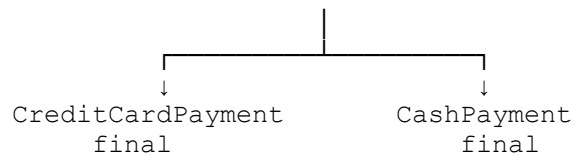
Now only the permitted classes can directly extend Payment.

```
public final class CreditCardPayment
    extends Payment {
}
```

```
public final class CashPayment
    extends Payment {
}
```

Conceptually:

Payment



An unauthorized class cannot extend it:

```
class CryptoPayment extends Payment {  
    // Compilation error  
}
```

2. Important Rules

A sealed class works together with permits and restrictions on its direct subclasses.

Rule	Explanation
sealed	Declares that inheritance is restricted
permits	Specifies the allowed direct subclasses
final	A permitted subclass can prevent further inheritance
sealed	A permitted subclass can continue restricting inheritance
non-sealed	A permitted subclass can reopen inheritance
Direct subclass	Must be explicitly permitted
Preview feature	Required preview compilation in Java 15

Example:

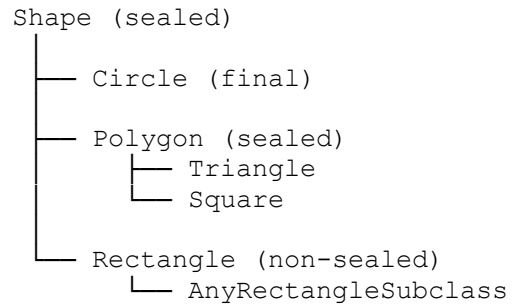
```
public sealed class Shape  
    permits Circle, Rectangle, Polygon {  
}
```

The permitted subclasses can have different inheritance policies:

```
public final class Circle extends Shape {  
}  
public sealed class Polygon extends Shape  
    permits Triangle, Square {  
}
```

```
public non-sealed class Rectangle extends Shape {  
}
```

This creates:



3. Advantages of Sealed Classes

Advantage	Explanation	Practical Benefit
Controlled inheritance	Explicitly defines who can extend a class	Prevents unwanted subclasses
Better domain modeling	Represents a closed set of valid types	Domain rules become clearer
Improved maintainability	Developers can understand the complete hierarchy	Easier code maintenance
Supports exhaustive reasoning	Compiler knows the permitted hierarchy	Useful with pattern matching
Better API design	Library authors control extension points	Prevents unsupported implementations
Improved encapsulation	Internal design can be protected from arbitrary extension	Reduces misuse
Makes invalid states harder to represent	Restricts possible type variations	Better correctness
Works well with pattern matching	Particularly useful with switch pattern matching	Cleaner type-based logic
Documents architecture	permits explicitly communicates intended	Self-documenting design

	subclasses	
Useful for finite domains	Excellent for concepts with a fixed number of variants	Payments, commands, shapes, states, etc.

4. Disadvantages of Sealed Classes

Disadvantage	Explanation	Impact
Restricts extensibility	New subclasses cannot be freely created	Can be problematic for extensible frameworks
More rigid architecture	The hierarchy must be planned in advance	Less flexible
Preview feature in Java 15	Required preview compilation/runtime options	Not ideal for production adoption on Java 15
Potential migration concerns	Java 15 code using the preview syntax needs consideration when moving versions	Additional migration work
Can over-constrain APIs	External users cannot create their own implementations	Limits library customization
More design decisions	Developers must decide which classes are permitted	Additional upfront design
Not suitable for open hierarchies	Plugin architectures often need arbitrary implementations	Sealed classes can be inappropriate
Can require hierarchy changes	Adding a new permitted subtype changes the central sealed declaration	More centralized maintenance
Inheritance remains somewhat complex	sealed, final, and non-sealed introduce additional concepts	Learning curve
Not a security mechanism	Restricting inheritance does not replace	Shouldn't be treated as security isolation

	authentication/authorization	
--	------------------------------	--

5. When to Use Sealed Classes

Situation	Use?	Reason
Fixed set of domain types	Yes	Perfect fit
Payment types	Yes	Known payment variants
Shapes	Yes	Finite set of shapes
Commands/events	Yes	Controlled event hierarchy
State machines	Yes	States can be explicitly restricted
AST nodes	Yes	Compiler knows valid node types
Result types	Yes	Success/failure variants
Protocol messages	Yes	Known message types
API response variants	Yes	Controlled response hierarchy
Security-sensitive domain modeling	Sometimes	Helps model allowed types, but isn't itself a security boundary
Public extensible framework	Usually no	Users may need to add implementations
Plugin architecture	Usually no	Plugins need arbitrary extension
Third-party implementations	Usually no	Restricts external subclasses
Frequently changing hierarchy	Usually no	Creates additional maintenance overhead
Simple inheritance	No	A normal class may be sufficient

6. Real-Time Use Case — Payment Processing

Suppose an application supports only these payment types:

```
public sealed interface Payment
    permits CreditCard, UPI, Cash {
}
```

Implementations:

```
public final class CreditCard implements Payment {
}
```

```
public final class UPI implements Payment {
}
```

```
public final class Cash implements Payment {
}
```

Now the application knows:

```
Payment
├── CreditCard
├── UPI
└── Cash
```

An accidental unsupported payment type cannot be introduced elsewhere:

```
class UnsupportedPayment implements Payment {
    // Compilation error
}
```

Why useful?

The business domain explicitly says:

These are the supported payment types.

The Java type system now communicates that rule.

7. Real-Time Use Case — Order State Machine

An e-commerce order might have a controlled set of states:

```
public sealed interface OrderState
    permits Created, Paid, Shipped, Delivered, Cancelled {
}
```

Then:

```
public final class Created implements OrderState {  
}  
  
public final class Paid implements OrderState {  
}  
  
public final class Shipped implements OrderState {  
}  
  
public final class Delivered implements OrderState {  
}  
  
public final class Cancelled implements OrderState {  
}
```

Conceptually:

```
OrderState  
├── Created  
├── Paid  
├── Shipped  
├── Delivered  
└── Cancelled
```

This is useful when the domain has a **finite, controlled lifecycle**.

8. Real-Time Use Case — API Result

A service can model its possible outcomes:

```
public sealed interface Result  
    permits Success, Failure {  
}  
public final class Success implements Result {  
    private final Object value;  
  
    public Success(Object value) {  
        this.value = value;  
    }  
}  
public final class Failure implements Result {  
    private final String message;  
  
    public Failure(String message) {
```

```
    this.message = message;
  }
}
```

This makes the API contract explicit:

Result

- Success
- Failure

Rather than allowing arbitrary implementations.

9. Real-Time Use Case — Compiler AST

Sealed hierarchies are especially useful in compiler and parser implementations.

For example:

```
public sealed interface Expression
    permits Literal, BinaryExpression, Variable {
}
```

Then:

Expression

- Literal
- BinaryExpression
- Variable

A compiler can reason about every supported expression type.

This is a natural use case because an AST often has a **known set of node categories**.

10. When NOT to Use Sealed Classes

Scenario	Why Avoid?	Better Choice
Plugin system	Plugins need to create new implementations	Interface
Public extension framework	External developers need extension points	Open interface/class

Frequently changing business model	New types are added regularly	Normal inheritance
Generic library API	Consumers may need custom implementations	Interface
Simple parent-child relationship	Sealing adds unnecessary complexity	Normal class
Dependency injection extension points	External implementations may be required	Interface
Strategy pattern with user-defined strategies	Number of implementations isn't fixed	Interface
Application with many independent modules	Central permits list can become restrictive	Normal hierarchy

11. Sealed Classes vs Alternatives

Sealed Classes	Alternative	When Alternative Is Better
sealed class	Normal class	When inheritance should remain open
sealed interface	Normal interface	When external implementations are expected
sealed + permits	final class	When there should be no subclasses at all
Sealed hierarchy	Enum	When variants are simple constants rather than different types
Sealed hierarchy	Abstract class	When subclasses should be unrestricted
Sealed hierarchy	Interface	When extension by external implementations is desirable
Sealed hierarchy	Records	When variants are primarily immutable data carriers
Sealed hierarchy	Visitor pattern	When operations across a closed hierarchy are the main concern

12. Sealed Class vs Enum

These can appear similar, but they solve different problems.

Aspect	Sealed Classes	Enum
Represents	Closed type hierarchy	Fixed set of constants
Each variant can have different fields	Yes	Yes, but within enum structure
Each variant can have its own class behavior	Yes	Yes, through enum-specific bodies
Supports inheritance hierarchy	Yes	No arbitrary class inheritance
Different subclasses	Yes	No
Best for	Different object types	Fixed named values
Example	Payment → UPI/Cash/Card	PaymentStatus → PENDING/PAID/FAILED

For example, use an enum for:

```
enum OrderStatus {  
    CREATED,  
    PAID,  
    SHIPPED,  
    CANCELLED  
}
```

But use a sealed hierarchy when each variant represents a **different type with potentially different structure and behavior**.

13. Sealed Classes + Pattern Matching

One of the strongest reasons to use sealed hierarchies is their relationship with pattern matching.

For example:

```

public sealed interface Shape
    permits Circle, Rectangle {
}
public final class Circle implements Shape {
}

public final class Rectangle implements Shape {
}

```

A pattern-matching switch can reason about the permitted variants:

```

static String describe(Shape shape) {
    return switch (shape) {
        case Circle c -> "Circle";
        case Rectangle r -> "Rectangle";
    };
}

```

The important idea is:

```

Sealed hierarchy
  ↓
Known permitted types
  ↓
Pattern matching
  ↓
More exhaustive type handling

```

This combination became particularly powerful as pattern matching features matured in later Java releases.

14. Java 15 Preview vs Java 17 Final

Aspect	Java 15	Java 17
Status	Preview	Final
JEP	JEP 360	JEP 409
sealed	Yes	Yes
permits	Yes	Yes
non-sealed	Yes	Yes
Restricted inheritance	Yes	Yes

Production recommendation	Caution	Recommended
Long-term use	Not ideal	Preferred

If you are maintaining a **Java 15** project, remember that SealedPreview is specifically a **preview feature**. For modern production applications, use the finalized sealed-class feature available from **Java 17 onward**.

15. Quick Decision Table

Requirement	Recommendation
Fixed number of related object types	Sealed class/interface
Fixed set of simple values	Enum
No inheritance allowed	Final class
Unlimited implementations	Interface
Unlimited subclasses	Non-final class
External developers must extend API	Normal interface/abstract class
Compiler/parser AST	Sealed hierarchy
Payment variants	Sealed hierarchy
Order states as simple values	Enum
Order states with different data/behavior	Sealed hierarchy
Plugin architecture	Normal interface
Java 15 preview project	Use cautiously
Java 17+ production project with a closed hierarchy	Sealed classes

Bottom line

SealedPreview in Java 15 is best when your domain has a deliberately closed set of related types. Its biggest advantage is that the inheritance rules become part of the Java type system rather than merely being documented conventions.

The biggest disadvantage is **loss of extensibility**. If consumers, plugins, or future modules should be able to create their own implementations, a normal interface or abstract class is generally a better choice.